

Boosting

Analítica de Datos



¿Qué es Boosting?

Boosting es una técnica de ensamble que combina múltiples modelos “débiles” para crear un modelo “fuerte”. A diferencia de Random Forest, que construye árboles independientes en paralelo, Boosting construye modelos secuencialmente.

Características Clave:

- Construcción secuencial de modelos
- Ponderación de observaciones
- Combinación ponderada de predicciones
- Enfoque en errores previos

AdaBoost (1995)

Desarrollado por Yoav Freund y Robert Schapire. Primer algoritmo de Boosting exitoso.

Características:

- Todas las observaciones inician con el mismo peso
- En cada iteración:
 - Se entrena un modelo débil
 - Se identifican errores
 - Se aumenta el peso de errores
 - Se disminuye el peso de aciertos
- Predicción final por promedio ponderado

Características y Ventajas:

- Utiliza el gradiente del error para identificar la dirección de máximo descenso
- Aplicable a cualquier función de pérdida diferenciable
- Más flexible que AdaBoost al no limitarse a clasificación binaria
- Mayor control sobre el proceso de aprendizaje

Parámetros Clave:

- `n_estimators`: Número de árboles
- `learning_rate`: Tasa de aprendizaje
- `max_depth`: Profundidad máxima
- `subsample`: Fracción de datos por árbol

Optimizaciones:

- Paralelización y optimización de memoria: Procesamiento más rápido usando múltiples núcleos
- Estructuras de datos especializadas: Formatos optimizados para mejor rendimiento
- Regularización L1 y L2 incorporada: Previene overfitting automáticamente
- Manejo automático de valores faltantes: Procesa datos incompletos sin preprocesamiento

Características Avanzadas:

- Early stopping inteligente: Monitorea el rendimiento y detiene el entrenamiento cuando no hay mejora
- Feature Importance: Proporciona métricas precisas sobre la contribución de cada variable
- Procesamiento optimizado de datos dispersos: Utiliza estructuras especializadas para matrices sparse
- Entrenamiento distribuido: Permite escalar el entrenamiento a múltiples núcleos de procesamiento

Comparación: Construcción

Característica	Random Forest	Boosting
Construcción	Paralela	Secuencial

- Random Forest: Árboles independientes contruidos simultáneamente
- Boosting: Modelos contruidos secuencialmente, cada uno corrigiendo errores del anterior

Comparación: Velocidad

Característica	Random Forest	Boosting
Velocidad	Más rápido	Más lento

- Random Forest: Paralelización natural, más rápido pero menos preciso
- Boosting: Secuencial, más lento pero potencialmente más preciso

Comparación: Overfitting

Característica	Random Forest	Boosting
Overfitting	Menos propenso	Más propenso

- Random Forest: Menor riesgo de overfitting por promediado
- Boosting: Mayor riesgo de overfitting por ajuste secuencial

Comparación: Ajuste

Característica	Random Forest	Boosting
Ajuste	Menos parámetros	Más parámetros

- Random Forest: Parámetros principales: `n_trees`, `max_depth`
- Boosting: Más parámetros: `learning_rate`, `n_estimators`, `subsample`, etc.

Comparación: Interpretabilidad

Característica	Random Forest	Boosting
Interpretabilidad	Mayor	Menor

- Random Forest: Importancia de variables más directa
- Boosting: Importancia de variables más compleja de interpretar

Implementación AdaBoost

```
1 from sklearn.ensemble import AdaBoostClassifier
2
3 ada_model = AdaBoostClassifier(
4     n_estimators=100,
5     learning_rate=1.0
6 )
```

Implementación Gradient Boosting

```
1 from sklearn.ensemble import GradientBoostingClassifier
2
3 gb_model = GradientBoostingClassifier(
4     n_estimators=100,
5     learning_rate=0.1,
6     max_depth=3
7 )
```

Implementación XGBoost

```
1 import xgboost as xgb
2
3 xgb_model = xgb.XGBClassifier(
4     n_estimators=100,
5     learning_rate=0.1,
6     max_depth=3,
7     reg_lambda=1,    # L2
8     reg_alpha=0      # L1
9 )
```

Entrenamiento y Predicción: como siempre

```
1 # Entrenamiento del modelo
2 model.fit(X_train, y_train)
3
4 # Prediccion
5 y_pred = model.predict(X_test)
```

- **Preprocesamiento:**
 - Escalar variables numéricas
 - Codificar variables categóricas
- **Validación:**
 - Validación cruzada
 - Ajuste de hiperparámetros
- **Optimización:**
 - Early stopping
 - Learning rate bajo (0.01-0.1)
 - Monitoreo de errores

¿Dudas?
¿Consultas?



Universidad de
SanAndrés